CO2 ASSIGNMENT

Precedence, Short-Circuit Evaluation and Bitwise Operators in C

Name	BAVADARANI S
Register Number	1912260562

Q1 - Precedence, Associativity and the Discount Bug (6 marks)

1.1 Code under analysis

```
total = price * qty - price * qty * 15 / 100 * senior + price * qty * 5 / 100;
net = total > 1000 ? total - 50 : total; /* Rs 50 off above Rs 1000 */
count = count++ + 1;
```

1.2 Rules used

- **Precedence (high to low):** * / (same level) > + - (same level) > > > ?: > =.
- **Associativity:** arithmetic operators group left-to-right; ?: and = group right-to-left.
- **Fully bracketed form of statement 1:** total = ((price*qty) - (((((price*qty)*15)/100)*senior))) + (((price*qty)*5)/100);
- Precedence and associativity fix only the **grouping**. The order in which independent operands (e.g. the three price*qty products) are evaluated is unspecified by C.

1.3 Predictions BEFORE running (all variables int)

Bill	price	qty	senior	Predicted total	Predicted net	Why
1	120	3	1	324	324	gross 360, discount 54, GST on 360 = 18; 324 is not > 1000
2	45	7	0	330	330	gross 315, discount 0, GST 315*5/100 = 15 (15.75 truncated)
3	500	3	1	1350	1300	gross 1500, discount 225, GST on 1500 = 75; 1350 > 1000 so minus 50

1.4 Operator-by-operator order (Bill 1: price=120, qty=3, senior=1)

Step	Operator applied	Computation	Result
1	* (price * qty), left product	120 * 3	360
2	* (... * 15)	360 * 15	5400
3	/ (... / 100)	5400 / 100	54
4	* (... * senior)	54 * 1	54
5	- (first term minus discount term)	360 - 54	306
6	* (price * qty) again, then * 5	360 * 5	1800
7	/ (... / 100)	1800 / 100	18
8	+ (left result + GST term)	306 + 18	324

Step	Operator applied	Computation	Result
9	= (right-to-left, assigns)	total = 324	total = 324

Steps 6 and 7 do not depend on steps 2-5, so a compiler may perform them earlier; only the dependencies in the parse tree are fixed. For the second statement: > is applied first (324 > 1000 gives 0); ?: then selects only the false branch total, so total - 50 is never evaluated; finally = stores 324 in net. For Bill 3: 1350 > 1000 is 1, so total - 50 = 1300 is evaluated and assigned.

1.5 Program and observed output

q1.c

```
#include <stdio.h>

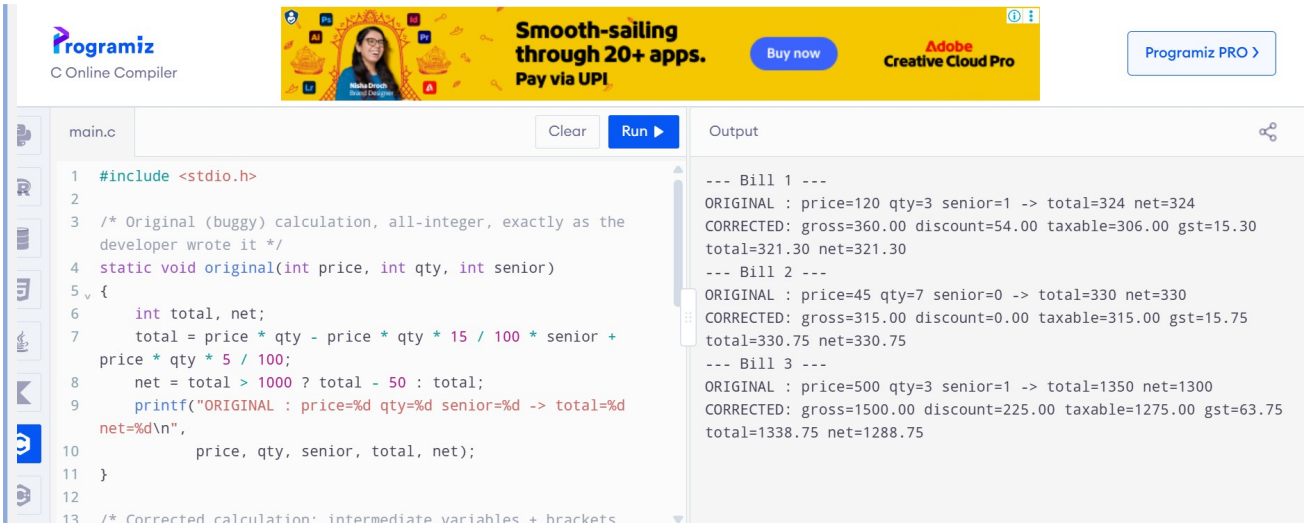
/* Original (buggy) calculation, all-integer, exactly as the developer wrote it */
static void original(int price, int qty, int senior)
{
    int total, net;
    total = price * qty - price * qty * 15 / 100 * senior + price * qty * 5 / 100;
    net = total > 1000 ? total - 50 : total;
    printf("ORIGINAL : price=%d qty=%d senior=%d -> total=%d net=%d\n",
           price, qty, senior, total, net);
}

/* Corrected calculation: intermediate variables + brackets, double arithmetic */
static void corrected(int price, int qty, int senior)
{
    double gross    = (double)price * qty;
    double discount = senior ? gross * 15.0 / 100.0 : 0.0;
    double taxable  = gross - discount;
    double gst      = taxable * 5.0 / 100.0;
    double total    = taxable + gst;
    double net      = (total > 1000.0) ? (total - 50.0) : total;
    printf("CORRECTED: gross=%.2f discount=%.2f taxable=%.2f gst=%.2f total=%.2f net=%.2f\n",
           gross, discount, taxable, gst, total, net);
}

int main(void)
{
    int bills[3][3] = { {120, 3, 1}, {45, 7, 0}, {500, 3, 1} };
    for (int i = 0; i < 3; i++) {
        printf("--- Bill %d ---\n", i + 1);
        original(bills[i][0], bills[i][1], bills[i][2]);
        corrected(bills[i][0], bills[i][1], bills[i][2]);
    }
    return 0;
}
```

Observed output

```
--- Bill 1 ---
ORIGINAL : price=120 qty=3 senior=1 -> total=324 net=324
CORRECTED: gross=360.00 discount=54.00 taxable=306.00 gst=15.30 total=321.30 net=321.30
--- Bill 2 ---
ORIGINAL : price=45 qty=7 senior=0 -> total=330 net=330
CORRECTED: gross=315.00 discount=0.00 taxable=315.00 gst=15.75 total=330.75 net=330.75
--- Bill 3 ---
ORIGINAL : price=500 qty=3 senior=1 -> total=1350 net=1300
CORRECTED: gross=1500.00 discount=225.00 taxable=1275.00 gst=63.75 total=1338.75 net=1288.75
```



Bill	Original total (net)	Intended total (net)	Difference	Prediction matched?
1	324 (324)	321.30 (321.30)	customer overcharged Rs 2.70	Yes
2	330 (330)	330.75 (330.75)	customer undercharged Rs 0.75	Yes
3	1350 (1300)	1338.75 (1288.75)	customer overcharged Rs 11.25	Yes

1.6 Expressions that differ from the pharmacist's intention

1. **GST computed on the wrong base (precedence/grouping bug).** The third term price*qty*5/100 taxes the gross amount, not the discounted amount. The intended rule is "(gross - discount) + 5% GST on that". Without brackets around gross - discount, the 5% is applied before the discount. This is why seniors are overcharged (Bills 1 and 3). For a non-senior (Bill 2) the grouping happens to be harmless.
2. **Integer division truncation.** price*qty*15/100 and price*qty*5/100 drop the fractional rupee: 315*5/100 = 15, not 15.75 (Bill 2 loses Rs 0.75). Since * and / are left-to-right, multiplying first and dividing last limits the loss to one truncation; writing price*qty*(15/100) would be far worse because 15/100 = 0 in integers and the discount would vanish completely.
3. * senior as a switch.** It works only if senior is exactly 0 or 1. A value such as 2 (or a character flag) would give a 30% discount. A conditional (senior ? ... : 0) states the intent.
4. **Second statement.** Its precedence is correct (> binds tighter than ?:, which binds tighter than =), but it receives the wrong, truncated total. At the boundary the error changes the outcome: a true payable of 1000.50 is stored as 1000, so the Rs 50 concession is wrongly refused.
5. **Third statement** count = count++ + 1; is undefined behaviour (see Inquiry).

1.7 Inquiry - why is `count = count++ + 1;` undefined behaviour?

In C (C11 6.5p2), if a side effect on a scalar object is unsequenced relative to another side effect on the same object, or to a value computation using that object, the behaviour is undefined. Here count is modified twice with no sequencing between the two writes: the post-increment count++ stores count+1 at some unspecified moment before the next sequence point, and the assignment = also stores a new value into count. Operators like = and + do not introduce a sequence point; the assignment only sequences its value computation (not the side effect of ++) before its own store. The language therefore allows any result (or any program behaviour), and different compilers or optimisation levels may legitimately differ. In general, an object must not be modified more than once, or modified and also read for another purpose, between two sequence points.

What the compiler does with -Wall. -Wall includes -Wsequence-point. gcc 13.3 printed:

```
q1ub.c: In function 'main':
q1ub.c:5:11: warning: operation on 'count' may be undefined [-Wsequence-point]
    5 |     count = count++ + 1;
      |     ~~~~~~^~~~~~~~~~~~~
```

It is only a warning: the program still compiles. With count = 5 it printed count = 6 at both -O0 and -O2 here, but that is an accident of this compiler, not a guarantee. A different statement such as i = i++ + 5 shows how results can vary. Using -Werror turns the warning into an error. The fix is simply count = count + 1; or count++; (one modification, no reuse).

1.8 Design decision - correct and readable rewrite

```
double gross    = (double)price * qty;                /* 1. price x quantity     */
double discount = senior ? gross * 15.0 / 100.0 : 0.0; /* 2. 15% for seniors only */
double taxable  = gross - discount;                   /* 3. amount after discount */
double gst      = taxable * 5.0 / 100.0;              /* 4. 5% GST on taxable     */
double total    = taxable + gst;                      /* 5. amount payable        */
double net      = (total > 1000.0) ? (total - 50.0) : total;  /* 6. Rs 50 off     */
count = count + 1;
```

Observed (corrected function in q1.c): Bill 1 total 321.30, Bill 2 total 330.75, Bill 3 total 1338.75 and net 1288.75 - exactly the intended values and different from the original in every bill.

Decision: use BOTH intermediate variables and brackets, but each only where it earns its place.

- Intermediate variables give each business rule (gross, discount, taxable, GST) its own named line, so the order of application is explicit, each rule can be checked or changed independently, and values can be inspected in a debugger.
- Brackets are used where precedence would otherwise not match intent, e.g. (total > 1000.0) ? (total - 50.0) : total and the (double)price * qty cast; the key grouping gross - discount is made impossible to miss by naming it taxable.
- A single bracketed expression such as total = (gross - gross*15/100*senior) * 105 / 100; would also be correct, but it hides the rules and is easy to break on the next edit.
- Money is computed in double here to show the result clearly; in real billing software the amounts should be kept in integer paise (e.g. long) with explicit rounding, to avoid both truncation and binary floating-point error.

1.9 Alternative concept and complexity analysis

Replacing the single long expression by several statements with temporaries (gross, discount, tax) gives the same arithmetic result as the corrected code above:

```
gross    = price * qty;
discount = gross * 15 / 100 * senior;   /* or the senior ? : form */
tax      = (gross - discount) * 5 / 100;
total    = gross - discount + tax;
```

Aspect	Single long expression	Intermediate variables
Operations (time)	10 arithmetic operations (price*qty computed 3 times: 1 + 4 + 3 + 2 add/sub); O(1)	9 operations (price*qty computed once); O(1). Difference is a tiny constant; an optimising compiler (common-subexpression elimination) produces nearly identical machine code for both.

Aspect	Single long expression	Intermediate variables
Extra space	None (compiler temporaries in registers)	3 extra variables (4 or 8 bytes each, tens of bytes): O(1), normally kept in registers anyway
Correctness	Easy to mis-group; the GST-on-gross bug hides in it	Each rule is a separate line, order is explicit, the bug cannot occur unnoticed, and each step is testable
Readability	Poor: one line mixing three rules	Good: self-documenting names, easy to maintain

Conclusion: the time complexity does not change (constant work per bill, O(1)), and the space cost is negligible, but correctness and readability improve substantially, so the intermediate-variable version is preferred.